Capítulo 2

1. **Which of the following Java operators can be used with boolean variables? (Choose all that apply.)**
   A. ==
   D. !
   G. Cast with (boolean)

   El operador == compara igualdad, ! es el complemento lógico y, aunque sea redundante, es legal realizar un cast de (boolean) a un valor que ya es booleano.

2. **What data type (or types) will allow the following code snippet to compile? (Choose all**

that apply.)

   byte apples = 5;

   short oranges = 10;

   _____ bananas = apples + oranges;

   A. int

   B. long

   D. double

   En Java, al sumar tipos pequeños (byte, short), el resultado se promueve automáticamente a int. Por tanto, puede almacenarse en un int, un long o un double.

3. **What change, when applied independently, would allow the following code snippet to compile? (Choose all that apply.)**
   3: long ear = 10;
   4: int hearin

   B. Cast ear on line 4 to int.
   C. Change the data type of ear on line 3 to short.
   D. Cast 2 * ear on line 4 to int.
   F. Change the data type of hearing on line 4 to long.

   Para asignar un long a un int se requiere un cast explícito o cambiar el tipo de la variable receptora a long para evitar la pérdida de precisión.

4. **What is the output of the following code snippet?**

```
3: boolean canine = true, wolf = true;
4: int teeth = 20;
5: canine = (teeth != 10) ^ (wolf=false);
6: System.out.println(canine+", "+teeth+", "+wolf);
```

B. true, 20, false

teeth != 10 es true. La expresión (wolf=false) asigna false y devuelve false. true ^ false resulta en true. Al final, teeth sigue siendo 20 y wolf ahora es false

5. **Which of the following operators are ranked in increasing or the same order of precedence? Assume the + operator is binary addition, not the unary form. (Choose all that apply.)**

A. +, *, %, --
C. =, ==, !

Los operadores están agrupados según su prioridad de ejecución (ej. los aritméticos antes que los de asignación).

6. **What is the output of the following program?**

```
1: public class CandyCounter {
2: static long addCandy(double fruit, float vegetables) {
3: return (int)fruit+vegetables;
4: }
5:
6: public static void main(String[] args) {
7: System.out.print(addCandy(1.4, 2.4f) + ", ");
8: System.out.print(addCandy(1.9, (float)4) + ", ");
9: System.out.print(addCandy((long)(int)(short)2, (float)4)); } }
```

F. None of the above.
El método realiza operaciones con float y double, y la conversión final a long o el manejo de decimales en el print no coincide con las opciones de respuesta típicas del examen.

7. **What is the output of the following code snippet?**
   int ph = 7, vis = 2;
   boolean clear = vis > 1 & (vis < 9 || ph < 2);
   boolean safe = (vis > 2) && (ph++ > 1);
   boolean tasty = 7 <= --ph;
   System.out.println(clear + "-" + safe + "-" + tasty);

   D. true-false-false
   clear es true. safe usa && (cortocircuito), como vis > 2 es false, no evalúa el resto y ph no aumenta. tasty evalúa el valor actual de ph tras el decremento.

8. **What is the output of the following code snippet?**
   4: int pig = (short)4;
   5: pig = pig++;
   6: long goat = (int)2;
   7: goat -= 1.0;
   8: System.out.print(pig + " - " + goat);

   A. 4 – 1

   pig = pig++ usa el valor original (4) y luego incrementa, pero la asignación sobreescribe el incremento, dejando a pig en 4. goat -= 1.0 convierte el resultado automáticamente a long (1).

9. **What are the unique outputs of the following code snippet? (Choose all that apply.)**
   int a = 2, b = 4, c = 2;
   System.out.println(a > 2 ? --c : b++);
   System.out.println(b = (a!=c ? a : b++));
   System.out.println(a > b ? b < c ? b : 2 : 1);

   A. 1
   D. 4
   E. 5

   Se evalúan las expresiones ternarias y los post-incrementos (b++), resultando en los valores 1, 4 y 5 en diferentes etapas.

10. **What are the unique outputs of the following code snippet? (Choose all that apply.)**

```
short height = 1, weight = 3;
 short zebra = (byte) weight * (byte) height;
 double ox = 1 + height * 2 + weight;
 long giraffe = 1 + 9 % height + 1;
 System.out.println(zebra);
 System.out.println(ox);
 System.out.println(giraffe);
```

G. The code does not compile.

Al multiplicar dos byte (o tras un cast a byte), el resultado se promueve a int. No se puede asignar un int a un short sin un cast explícito, por lo que el código no compila.

11. **What is the output of the following code?**

```
11: int sample1 = (2 * 4) % 3;
12: int sample2 = 3 * 2 % 3;
13: int sample3 = 5 * (1 % 2);
14: System.out.println(sample1 + ", " + sample2 + ", " + sample3);
```

D. 2, 0, 5
Se sigue el orden de operaciones: (8)%3 = 2, 6%3 = 0, y 5*(1) = 5.

12. **The _________ operator increases a value and returns the original value, while the _______ operator decreases a value and returns the new value.**

D. post-increment, pre-decrement

El post-incremento devuelve el valor original antes de sumar; el pre-decremento resta primero y devuelve el valor nuevo.

13. **What is the output of the following code snippet?**

```
boolean sunny = true, raining = false, sunday = true;
boolean goingToTheStore = sunny & raining ^ sunday;
boolean goingToTheZoo = sunday && !raining;
boolean stayingHome = !(goingToTheStore && goingToTheZoo);
System.out.println(goingToTheStore + "-" + goingToTheZoo
+ "-" +stayingHome);
```

F. true-true-false

Se evalúan los operadores lógicos &, ^ y && siguiendo su tabla de verdad, resultando en true-true-false.

14. **Which of the following statements are correct? (Choose all that apply.)**
    B. The inequality operator (!=) can be used to compare objects.
    E. The return value of an assignment operation expression is the value of the newly assigned variable.
    G. The logical complement operator (!) cannot be used to flip numeric values.

    != funciona con objetos (compara referencias). El resultado de una asignación es el valor asignado. ! es exclusivo para booleanos.

15. **Which operators take three operands or values? (Choose all that apply.)**
    D. ? :

    El único operador que toma tres operandos es el condicional ? :.

16. **How many lines of the following code contain compiler errors?**

```
int note = 1 * 2 + (long)3;
short melody = (byte)(double)(note *= 2);
double song = melody;
float symphony = (float)((song == 1_000f) ? song * 2L : song);
```

    B. 1

    La línea de note intenta asignar una expresión que resulta en long a una variable int sin cast, lo cual es un error de compilación.

17. **Given the following code snippet, what are the values of the variables after it is executed?**
    (Choose all that apply.)

```
int ticketsTaken = 1;
int ticketsSold = 3;
ticketsSold += 1 + ticketsTaken++;
ticketsTaken *= 2;
ticketsSold += (long)1;
```

    C. ticketsSold is 6.
    F. ticketsTaken is 4.

    Siguiendo el flujo: ticketsTaken termina en 4 tras el incremento y la multiplicación. ticketsSold suma los valores acumulados hasta llegar a 6.

18. **Which of the following can be used to change the order of operation in an expression?**
    (Choose all that apply.)

    C. ( )

    Los paréntesis ( ) tienen la prioridad más alta y se usan para alterar el orden natural de evaluación.

19. **What is the result of executing the following code snippet? (Choose all that apply.)**

    3: int start = 7;
    4: int end = 4;
    5: end += ++start;
    6: start = (byte)(Byte.MAX_VALUE + 1);

    B. start is -128.
    F. end is 12
    Byte.MAX_VALUE + 1 (127 + 1) resulta en -128 al hacer el cast a byte. end acumula el valor previo de start.

20. **Which of the following statements about unary operators are true? (Choose all that apply.)**

    A. Unary operators are always executed before any surrounding numeric binary or ternary operators.

    D. The post-decrement operator (--) returns the value of the variable before the decrement is applied.

    E. The ! operator cannot be used on numeric values.

    Los unarios tienen alta precedencia, el post-decremento devuelve el valor previo, y ! no se aplica a números.

21. **What is the result of executing the following code snippet?**

```
int myFavoriteNumber = 8;
 int bird = ~myFavoriteNumber;
 int plane = -myFavoriteNumber;
 var superman = bird == plane ? 5 : 10;
 System.out.println(bird + "," + plane + "," + --superman);
```

E. -9,-8,9

~8 es -9. -8 es -8. La comparación es false, por lo que el ternario elige 10, y el pre-decremento --10 imprime 9.

Capítulo 3

1. **Which of the following data types can be used in a switch expression? (Choose all that apply.)**

A. enum
B. int
C. Byte
E. String
F. char
G. var

Java permite int (y sus wrappers), String, enum, char y var (si se resuelve a un tipo soportado).

2. **What is the output of the following code snippet? (Choose all that apply.)**

```
 3: int temperature = 4;
 4: long humidity = -temperature + temperature * 3;
 5: if (temperature>=4)
 6: if (humidity < 6) System.out.println("Too Low");
 7: else System.out.println("Just Right");
 8: else System.out.println(("Too High");
```

B. Just Right

humidity resulta en 8. Como temperature >= 4 es true pero humidity < 6 es false, se ejecuta el else de la línea 7 ("Just Right").

3. Which of the following data types are permitted on the right side of a for-each expression? Choose all that apply.)

A. Double[][]
D. List
F. char[]
H. Set

Se pueden iterar arreglos y cualquier clase que implemente Iterable (como List y Set).

4. What is the output of calling printReptile(6)?

```
void printReptile(int category) {
var type = switch(category) {
case 1,2 -> "Snake";
case 3,4 -> "Lizard";
case 5,6 -> "Turtle";
case 7,8 -> "Alligator";
};
System.out.print(type);
}
```

F. None of the above

Las expresiones switch deben ser exhaustivas. Si no cubren todos los valores posibles (o falta un default), el código no compila.

5. What is the output of the following code snippet?

```
List<Integer> myFavoriteNumbers = new ArrayList<>();
myFavoriteNumbers.add(10);
myFavoriteNumbers.add(14);
for (var a : myFavoriteNumbers) {
System.out.print(a + ", ");
break;
}
for (int b : myFavoriteNumbers) {
continue;
System.out.print(b + ", ");
}
for (Object c : myFavoriteNumbers)
System.out.print(c + ", ");
```

E. Exactly one line of code does not compile.

La línea con continue seguida de un System.out.print es código inalcanzable (unreachable code), lo que genera un error de compilación.

6. **Which statements about decision structures are true? (Choose all that apply.)**

   C. The conditional expression of a for loop is evaluated before the first execution of the loop body.
   D. A switch expression that takes a String and assigns the result to a variable requires a
   default branch.
   E. The body of a do/while loop is guaranteed to be executed at least once.

   El do/while siempre corre una vez. Un switch que asigna valor requiere default si no es exhaustivo. El for evalúa la condición antes de entrar.

7. **Assuming weather is a well-formed nonempty array, which code snippet, when inserted independently into the blank in the following code, prints all of the elements of weather? (Choose all that apply.)**

```
private void print(int[] weather) {
 for( ) {
 System.out.println(weather[i]);
 }
 }
```

   B. int i=0; i<=weather.length-1; ++i
   D. int i=weather.length-1; i>=0; i—

   Muestran la sintaxis correcta para recorrer un arreglo desde el índice 0 hasta length-1 o viceversa.

8. **What is the output of calling printType(11)?**

```
void printType(Object o) {
if(o instanceof Integer bat) {
System.out.print("int");
} else if(o instanceof Integer bat && bat < 10) {
System.out.print("small int");
} else if(o instanceof Long bat || bat <= 20) {
System.out.print("long");
} default {
System.out.print("unknown");
}
}
```

G. The code contains two lines that do not compile.

Hay dos errores: el uso de la palabra default en un bloque if/else no es válido, y la variable bat está fuera de alcance o redefinida incorrectamente.

9. Which statements, when inserted independently into the following blank, will cause the code to print 2 at runtime? (Choose all that apply.)

```
int count = 0;
BUNNY: for(int row = 1; row <=3; row++)
RABBIT: for(int col = 0; col <3 ; col++) {
if((col + row) % 2 == 0)
;
count++;
}
System.out.println(count);
```

B. break RABBIT
C. continue BUNNY
E. break

Los comandos de salto (break, continue) con etiquetas permiten salir o saltar niveles de bucles anidados para detener el contador en 2.

10. Given the following method, how many lines contain compilation errors? (Choose all that apply.)

```
10: private DayOfWeek getWeekDay(int day, final int thursday) {
11: int otherDay = day;
12: int Sunday = 0;
13: switch(otherDay) {
14: default:
15: case 1: continue;
16: case thursday: return DayOfWeek.THURSDAY;
17: case 2,10: break;
18: case Sunday: return DayOfWeek.SUNDAY;
19: case DayOfWeek.MONDAY: return DayOfWeek.MONDAY;
20: }
21: return DayOfWeek.FRIDAY;
22: }
```

E. 4

Tiene 4 errores, incluyendo el uso de continue fuera de un bucle y etiquetas de case que no son constantes en tiempo de compilación.

11. What is the output of calling printLocation(Animal.MAMMAL)?

```
10: class Zoo {
 11: enum Animal {BIRD, FISH, MAMMAL}
 12: void printLocation(Animal a) {
 13: long type = switch(a) {
 14: case BIRD -> 1;
 15: case FISH -> 2;
 16: case MAMMAL -> 3;
 17: default -> 4;
 18: };
 19: System.out.print(type);
 20: } }
```

A. 3

El switch detecta el caso MAMMAL y devuelve el valor 3.

12. What is the result of the following code snippet?

```
 3: int sing = 8, squawk = 2, notes = 0;
 4: while(sing > squawk) {
 5: sing--;
 6: squawk += 2;
7: ing + squawk;
 8: }
 9: System.out.println(notes);
```

C. 23

El bucle se ejecuta mientras sing > squawk, acumulando valores en la lógica interna hasta que la condición falla.

13. What is the output of the following code snippet?

```
 2: boolean keepGoing = true;
 3: int result = 15, meters = 10;
4: do {
 5: meters--;
 6: if(meters==8) keepGoing = false;
 7: result -= 2;
 8: } while keepGoing;
 9: System.out.println(result)
```

G. The code does not compile for a different reason.

La condición del while en un bloque do/while debe ir obligatoriamente entre paréntesis: while (keepGoing).

14. Which statements about the following code snippet are correct? (Choose all that apply.)

```
for(var penguin : new int[2])
System.out.println(penguin);
var ostrich = new Character[3];
for(var emu : ostrich)
System.out.println(emu);
List<Integer> parrots = new ArrayList<Integer>();
for(var macaw : parrots)
System.out.println(macaw)
```

B. The data type of penguin is int.
D. The data type of emu is Character.
F. The data type of macaw is Integer

En un for-each, var infiere el tipo de los elementos del arreglo o colección (ej. int para int[]).

15. What is the result of the following code snippet?

```
final char a = 'A', e = 'E';
char grade = 'B';
switch (grade) {
default:
case a:
case 'B': 'C': System.out.print("great ");
case 'D': System.out.print("good "); break;
case e:
case 'F': System.out.print("not good ");
}
```

F. None of the above

La sintaxis case 'B': 'C': es incorrecta para un switch tradicional; debería ser case 'B', 'C': o casos separados.

16. **Given the following array, which code snippets print the elements in reverse order from how they are declared? (Choose all that apply.)**
    char[] wolf = {'W', 'e', 'b', 'b', 'y'};

    A.
    ```
    int q = wolf.length;
    for( ; ; ) {
    System.out.print(wolf[--q]);
    if(q==0) break;
    }
    ```
    B.
    ```
    for(int m=wolf.length-1; m>=0; --m)
    System.out.print(wolf
    ```
    D.
    ```
    int x = wolf.length-1;
    for(int j=0; x>=0 && j==0; x--)
    System.o
    ```

    Son formas válidas de manipular los índices desde length-1 hasta 0.

17. **What distinct numbers are printed when the following method is executed? (Choose all**
    that apply.)

```
private void countAttendees() {
int participants = 4, animals = 2, performers = -1;
while((participants = participants+1) < 10) {}
do {} while (animals++ <= 1);
for( ; performers<2; performers+=2) {}
System.out.println(participants);
System.out.println(animals);
System.out.println(performers);
}
```

B. 3
E. 10

Tras evaluar el while, el do/while y el for, los valores finales de las variables son 10 y 3.

18. Which statements about pattern matching and flow scoping are correct? (Choose all
that apply.)

C. Pattern matching with an if statement is implemented using the instanceof operator.
E. Flow scoping means a pattern variable is only accessible if the compiler can discern its type.

El "flow scoping" permite que la variable del patrón solo sea accesible donde el compilador garantiza que el tipo coincide.

19. What is the output of the following code snippet?

```
2 double iguana = 0;
3: do {
4: int snake = 1;
5: System.out.print(snake++ + " ");
6: iguana--;
7: } while (snake <= 5);
8: System.out.println(iguana);
```

E. The code does not compile.

La variable snake se declara dentro del bloque do. La condición del while está fuera de ese bloque, por lo que no puede ver a snake.

20. Which statements, when inserted into the following blanks, allow the code to compile and run without entering an infinite loop? (Choose all that apply.)

```
4: int height = 1;
5: L1: while(height++ <10) {
6: long humidity = 12;
7: L2: do {
8: if(humidity-- % 12 == 0) ;
9: int temperature = 30;
10: L3: for( ; ; ) {
11: temperature++;
12: if(temperature>50) ;
13: }
14: } while (humidity > 4);
```

A. break L2 on line 8; continue L2 on line 12
E. continue L2 on line 8; continue L2 on line 12

El uso de break o continue con etiquetas permite romper el bucle infinito for(; ;).

21. **A minimum of how many lines need to be corrected before the following method will compile?**

```
21: void findZookeeper(Long id) {
22: System.out.print(switch(id) {
23: case 10 -> {"Jane"}
24: case 20 -> {yield "Lisa";};
25: case 30 -> "Kelly";
26: case 30 -> "Sarah";
27: default -> "Unassigned";
28: });
29: }
```

E. Four

Requiere corregir 4 líneas, incluyendo casos duplicados (case 30) y el uso de llaves incorrectas en las flechas del switch.

22. **What is the output of the following code snippet? (Choose all that apply.)**

```
2: var tailFeathers = 3;
3: final var one = 1;
4: switch (tailFeathers) {
5: case one: System.out.print(3 + " ");
6: default: case 3: System.out.print(5 + " ");
7: }
8: while (tailFeathers > 1) {
9: System.out.print(--tailFeathers + " "); }
```

E. 5 2 1

Entra en case 3 (imprime 5) y luego el bucle while imprime el decremento 2 1.

23. **What is the output of the following code snippet?**

```
15: int penguin = 50, turtle = 75;
16: boolean older = penguin >= turtle;
17: if (older = true) System.out.println("Success");
18: else System.out.println("Failure");
19: else if(penguin != 50) System.out.println("Other");
```

F. None of the above

El código tiene un else huérfano después de otro else, lo cual es un error de sintaxis. Además, older = true es una asignación, no una comparación.

24. **Which of the following are possible data types for friends that would allow the code to compile? (Choose all that apply.)**

```
for(var friend in friends) {
System.out.println(friend);
}
```

G. None of the above

Java no utiliza la palabra clave in para los bucles; utiliza el símbolo de dos puntos :.

25. **What is the output of the following code snippet?**

```
6: String instrument = "violin";
7: final String CELLO = "cello";
8: String viola = "viola";
9: int p = -1;
10: switch(instrument) {
11: case "bass" : break;
12: case CELLO : p++;
13: default: p++;
14: case "VIOLIN": p++;
15: case "viola" : ++p; break;
16: }
17: System.out.print(p);
```

D. 2

El switch es sensible a mayúsculas. "violin" no coincide con "VIOLIN", por lo que cae en el default e incrementa p hasta 2 debido a la falta de break.

26. **What is the output of the following code snippet? (Choose all that apply.)**

```
9: int w = 0, r = 1;
10: String name = "";
11: while(w < 2) {
12: name += "A";
13: do {
14: name += "B";
15: if(name.length()>0) name += "C";
16: else break;
17: } while (r <=1);
18: r++; w++; }
19: System.out.println(name);
```

F. The code compiles but never terminates at runtime

El bucle do/while interno no tiene una condición que cambie a false de forma efectiva, provocando que el programa no termine.

27. What is printed by the following code snippet?

```
23: byte amphibian = 1;
24: String name = "Frog";
25: String color = switch(amphibian) {
26: case 1 -> { yield "Red"; }
27: case 2 -> { if(name.equals("Frog")) yield "Green"; }
28: case 3 -> { yield "Purple"; }
29: default -> throw new RuntimeException();
30: };
31: System.out.print(color);
```

F. The code does not compile.

En la línea 27, el bloque if no tiene un else que devuelva un valor mediante yield. Todas las rutas de un bloque de expresión switch deben devolver un valor.

28. What is the output of calling getFish("goldie")?

```
void getFish(Object fish) {
if (!(fish instanceof String guppy))
System.out.print("Eat!");
else if (!(fish instanceof String guppy)) {
throw new RuntimeException();
}
System.out.print("Swim!");
}
```

F. None of the above

No compila porque intenta declarar la variable de patrón guppy dos veces en el mismo ámbito de flujo.

29. What is the result of the following code?

```
public class PrintIntegers {
public static void main(String[] args) {
int y = -2;
do System.out.print(++y + " ");
while(y <= 5);
} }
```

C. -1 0 1 2 3 4 5 6

El bucle do incrementa y antes de imprimir, empezando en -1 y terminando en 6 cuando la condición y <= 5 se evalúa tras imprimir el 6.